

CDAC Admission Process Case Study

Design database for CDAC Admission System.

Create appropriate tables and insert sample records. Write a Stored Procedure for Round1 allocation.

Write a Stored Procedure for Round2 allocation.

Note: 1. Create ER diagram following Normalization process.

2. Study database constraints in depth: PRIMARY KEY, FOREIGN-KEY, UNIQUE, CHECK, DEFAULT Apply appropriate constraints in CDAC admission system.

3. Learn about composite keys and candidate keys Identify keys in CDAC admission system.

4. Explore AUTO_INCREMENT / SEQUENCES Use appropriate AUTO_INCREMENT keys in CDAC admission system.

5. Study control structures: IF-ELSE, CASE, LOOPS (WHILE, REPEAT, LOOP) Use appropriate control structures in CDAC admission system stored procedures.

6. Learn stored procedure parameters: IN, OUT, IN-OUT. Use appropriate parameters for CDAC admission system stored procedures.

7. Understand cursors and their usage in following stored procedures. Make appropriate use of cursor in CDAC admission system stored procedures.

8. Explore error handling in stored procedures Implement appropriate error handling in CDAC admission system stored procedures

```
KD4-98876-Anuradha> CREATE TABLE Student (
->     student_id INT AUTO_INCREMENT PRIMARY KEY,
->     student_name VARCHAR(100) NOT NULL,
->     email VARCHAR(100) NOT NULL UNIQUE,
->     mobile VARCHAR(15) NOT NULL UNIQUE,
->     ccet_rank INT NOT NULL UNIQUE,
->     category VARCHAR(20) DEFAULT 'OPEN',
->     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->
->     CHECK (ccet_rank > 0),
->     CHECK (category IN ('OPEN', 'EWS', 'OBC', 'SC', 'ST')))
-> );
```

```
KD4-98876-Anuradha> ^C
KD4-98876-Anuradha> ^C
KD4-98876-Anuradha> CREATE TABLE Course (
->     course_id INT AUTO_INCREMENT PRIMARY KEY,
->     course_name VARCHAR(100) NOT NULL UNIQUE,
->     duration_months INT NOT NULL DEFAULT 6,
->     total_fees DECIMAL(10,2) NOT NULL,
->
->     CHECK (duration_months > 0),
->     CHECK (total_fees > 0)
-> );
Query OK, 0 rows affected (0.278 sec)
```

use of
8. Ex
CDAC

KD4-9
-
-
-
-
-

```

KD4-98876-Anuradha> CREATE TABLE Course_Center (
->     course_center_id INT AUTO_INCREMENT PRIMARY KEY,
->     course_id INT NOT NULL,
->     center_id INT NOT NULL,
->     available_seats INT NOT NULL,
->
->     FOREIGN KEY (course_id) REFERENCES Course(course_id),
->     FOREIGN KEY (center_id) REFERENCES Center(center_id),
->
->     UNIQUE (course_id, center_id),
->     CHECK (available_seats >= 0)
-> );
Query OK, 0 rows affected (0.305 sec)

```

```

KD4-98876-Anuradha> UNIQUE (course_id, center_id)
-> ^C
KD4-98876-Anuradha> CREATE TABLE Preference (
->     preference_id INT AUTO_INCREMENT PRIMARY KEY,
->     student_id INT NOT NULL,
->     course_center_id INT NOT NULL,
->     preference_no INT NOT NULL,
->
->     FOREIGN KEY (student_id) REFERENCES Student(student_id),
->     FOREIGN KEY (course_center_id) REFERENCES Course_Center(course_center_id),
->
->     UNIQUE (student_id, preference_no),
->     UNIQUE (student_id, course_center_id),
->
->     CHECK (preference_no > 0)
-> );
Query OK, 0 rows affected (0.264 sec)

```

```

KD4-98876-Anuradha> CREATE TABLE Admission_Round (
->     round_id INT AUTO_INCREMENT PRIMARY KEY,
->     round_name VARCHAR(20) NOT NULL UNIQUE,
->
->     CHECK (round_name IN ('ROUND1', 'ROUND2'))
-> );
Query OK, 0 rows affected (0.324 sec)

KD4-98876-Anuradha> INSERT INTO Admission_Round (round_name)
-> VALUES
-> ('ROUND1'),
-> ('ROUND2');
Query OK, 2 rows affected (0.051 sec)
Records: 2 Duplicates: 0 Warnings: 0

KD4-98876-Anuradha> CREATE TABLE Allocation (
->     allocation_id INT AUTO_INCREMENT PRIMARY KEY,
->     student_id INT NOT NULL,
->     course_center_id INT NOT NULL,
->     round_id INT NOT NULL,
->     preference_no INT NOT NULL,
->     allocation_status VARCHAR(20) DEFAULT 'ALLOCATED',
->     allocation_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->
->     FOREIGN KEY (student_id) REFERENCES Student(student_id),
->     FOREIGN KEY (course_center_id) REFERENCES Course_Center(course_center_id),
->     FOREIGN KEY (round_id) REFERENCES Admission_Round(round_id),
->
->     UNIQUE (student_id, round_id),
->
->     CHECK (allocation_status IN ('ALLOCATED', 'FREEZED', 'WITHDRAWN'))
-> );
Query OK, 0 rows affected (0.293 sec)

```

```

KD4-98876-Anuradha> INSERT INTO Student
-> (student_name, email, mobile, ccet_rank, category)
-> VALUES
-> ('Amit Patil', 'amit@gmail.com', '90000000001', 100, 'OPEN'),
-> ('Sneha Joshi', 'sneha@gmail.com', '90000000002', 250, 'OPEN'),
-> ('Rahul Sharma', 'rahul@gmail.com', '90000000003', 400, 'OBC'),
-> ('Pooja Deshmukh', 'pooja@gmail.com', '90000000004', 600, 'EWS'),
-> ('Neha Kulkarni', 'neha@gmail.com', '90000000005', 800, 'OPEN'),
-> ('Akash More', 'akash@gmail.com', '90000000006', 1000, 'SC');
Query OK, 6 rows affected (0.036 sec)
Records: 6 Duplicates: 0 Warnings: 0

```

```

KD4-98876-Anuradha> DELIMITER $$
KD4-98876-Anuradha>
KD4-98876-Anuradha> CREATE PROCEDURE Round1_Allocation(
->     OUT p_allocated INT,
->     OUT p_message VARCHAR(200)
-> )
-> BEGIN
->
->     DECLARE v_done INT DEFAULT 0;
->     DECLARE v_student_id INT;
->     DECLARE v_course_center_id INT;
->     DECLARE v_preference_no INT;
->     DECLARE v_seat INT DEFAULT 0;
->     DECLARE v_allocated_count INT DEFAULT 0;
->
->     DECLARE student_cursor CURSOR FOR
->         SELECT student_id
->         FROM Student
->         ORDER BY ccet_rank;
->
->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done =
->
->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
->     BEGIN
->         ROLLBACK;
->         SET p_allocated = 0;
->         SET p_message = 'Error occurred during Round 1 a
->     END;
->
->     START TRANSACTION;
->
->     OPEN student_cursor;
->
->     student_loop: LOOP
->
->         FETCH student_cursor INTO v_student_id;
->
->         IF v_done = 1 THEN
->             LEAVE student_loop;

```

```

SELECT available_seats
INTO v_seat
FROM Course_Center
WHERE course_center_id = v_course_center_id
FOR UPDATE;

IF v_seat > 0 THEN

    INSERT INTO Allocation
    (
        student_id,
        course_center_id,
        round_id,
        preference_no,
        allocation_status
    )
    VALUES
    (
        v_student_id,
        v_course_center_id,
        1,
        v_preference_no,
        'ALLOCATED'
    );

    UPDATE Course_Center
    SET available_seats = available_seats - v_seat
    WHERE course_center_id = v_course_center_id;

    SET v_allocated_count =
        v_allocated_count + 1;

    LEAVE preference_loop;

END IF;

```

```

->
->          SET v_allocated_count =
->              v_allocated_count + 1;
->
->          LEAVE preference_loop;
->
->      END IF;
->
->  END LOOP;
->
->      CLOSE pref_cursor;
->
->  END;
->
->  END LOOP;
->
->  CLOSE student_cursor;
->
->  COMMIT;
->
->  SET p_allocated = v_allocated_count;
->
->  SET p_message =
->      'Round 1 allocation completed successfully';
->
->  END$$
Query OK, 0 rows affected (0.173 sec)

```

```

->
->  END$$
Query OK, 0 rows affected (0.173 sec)

KD4-98876-Anuradha>
KD4-98876-Anuradha> DELIMITER ;
KD4-98876-Anuradha> CALL Round1_Allocation(@count, @message);
Query OK, 0 rows affected (0.075 sec)

KD4-98876-Anuradha>
KD4-98876-Anuradha> SELECT @count AS Allocated_Students,
->      @message AS Message;
+-----+-----+
| Allocated_Students | Message |
+-----+-----+
| 0 | Round 1 allocation completed successfully |
+-----+-----+
1 row in set (0.003 sec)

```

```
KD4-98876-Anuradha> DELIMITER $$
KD4-98876-Anuradha>
KD4-98876-Anuradha> CREATE PROCEDURE Round2_Allocation(
->     OUT p_allocated INT,
->     OUT p_message VARCHAR(200)
-> )
-> BEGIN
->
->     DECLARE v_done INT DEFAULT 0;
->     DECLARE v_student_id INT;
->     DECLARE v_course_center_id INT;
->     DECLARE v_preference_no INT;
->     DECLARE v_seat INT DEFAULT 0;
->     DECLARE v_allocated_count INT DEFAULT 0;
->
```

```

DECLARE student_cursor CURSOR FOR
    SELECT s.student_id
    FROM Student s
    WHERE NOT EXISTS
    (
        SELECT 1
        FROM Allocation a
        WHERE a.student_id = s.student_id
              AND a.round_id = 1
    )
    ORDER BY s.ccet_rank;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    SET p_allocated = 0;
    SET p_message =
        'Error occurred during Round 2 allocation';
END;

START TRANSACTION;

OPEN student_cursor;

student_loop: LOOP

    FETCH student_cursor INTO v_student_id;

    IF v_done = 1 THEN
        LEAVE student_loop;
    END IF;

    BEGIN

```

```
-> BEGIN
->
-> DECLARE pref_done INT DEFAULT 0;
->
-> DECLARE pref_cursor CURSOR FOR
->     SELECT course_center_id, preference_no
->     FROM Preference
->     WHERE student_id = v_student_id
->     ORDER BY preference_no;
->
-> DECLARE CONTINUE HANDLER FOR NOT FOUND
->     SET pref_done = 1;
->
-> OPEN pref_cursor;
->
-> preference_loop: LOOP
->
->     FETCH pref_cursor
->     INTO v_course_center_id,
->         v_preference_no;
->
->     IF pref_done = 1 THEN
->         LEAVE preference_loop;
->     END IF;
->
->     SELECT available_seats
->     INTO v_seat
->     FROM Course_Center
->     WHERE course_center_id =
->         v_course_center_id
->     FOR UPDATE;
->
->     IF v_seat > 0 THEN
```

```

->
->          END IF;
->
->          END LOOP;
->
->          CLOSE pref_cursor;
->
->      END;
->
->  END LOOP;
->
->  CLOSE student_cursor;
->
->  COMMIT;
->
->  SET p_allocated = v_allocated_count;
->
->  SET p_message =
->      'Round 2 allocation completed successfully';
->
-> END$$
Query OK, 0 rows affected (0.095 sec)

```

```

KD4-98876-Anuradha>
KD4-98876-Anuradha> DELIMITER ;
KD4-98876-Anuradha> CALL Round2_Allocation(@count, @message);
Query OK, 0 rows affected (0.091 sec)

```

```

KD4-98876-Anuradha>
KD4-98876-Anuradha> SELECT @count AS Allocated_Students,
->      @message AS Message;
+-----+-----+
| Allocated_Students | Message |
+-----+-----+
| 0 | Round 2 allocation completed successfully |
+-----+-----+
1 row in set (0.003 sec)

```